Colegio Tecnológico UTN · Diseño Backend · 5.° año ·

Práctica — Lanzamiento del TP: la base de tu tienda

Unidad 7 · Clase 1 · Consigna, grupos y rubros · Proyecto, tablas e imágenes en Supabase

Hoy arranca el TP integrador: una tienda online completa, hecha con Express, Supabase y JavaScript, y publicada en Vercel. En la primera parte de la clase conocés la consigna, armás tu grupo y te toca un rubro. En la segunda, creás la base de datos de tu tienda con los productos de tu rubro y sus fotos, y dejás la plantilla del proyecto corriendo en tu compu.

Usamos una librería como ejemplo. Tu grupo aplica los mismos pasos con el rubro que le tocó.

El TP integrador — La consigna

Tu tienda tiene que cumplir **todo** este mínimo para aprobar:

Catálogo	mostrar los productos con foto, precio y stock, con filtro por categoría
Carrito	agregar, quitar y cambiar cantidades, sin perderse al recargar la página
Usuarios	registro y login con Supabase Auth
Compra	el backend controla el stock y calcula el total con los precios de la base
Online	publicada en Vercel, con un link que funcione

Una vez que el mínimo funciona y está online, podés sumar **extras** para mejorar la nota:

Panel admin	ABM de productos protegido con rol admin: un cliente recibe 403
Buscador	buscar productos por nombre
Mis pedidos	cada usuario ve sus compras con sus items
Detalle de producto	una página por producto, con 404 si no existe

> ⚠ **Regla de oro: primero el mínimo funcionando y online.**
> Una tienda simple que anda en internet vale más que una brillante que no funciona. Nadie arranca un extra hasta tener el mínimo andando.

Grupos y rubros

Se trabaja en **grupos de 2**. Cada grupo tiene un rubro distinto, que se sortea en clase y no se repite en la comisión. Si tu grupo quiere proponer un rubro propio, se acepta siempre que tenga categorías y productos claros.

Librería	Vivero	Juguetería
Ferretería	Café en grano	Mascotas
Camping	Instrumentos musicales	Cosmética natural
Bicicletería	Ropa deportiva	Panadería

1		
2		
3		
4		
5		
6		

> ⚠ **Todos programan.**
> En la defensa, cualquier integrante puede tener que explicar cualquier parte del código. Repartan el trabajo, pero entiendan todo.

El camino hasta la presentación

1	Consigna, grupos y la base de la tienda (hoy)	
2	Usuarios, catálogo y pedidos en el backend	
3	El front: catálogo, carrito, login y compra con fetch	
4	Deploy en Vercel	**Hito 1:** tienda online con el backend completo
5	Taller: completar el mínimo	
6	Taller y pruebas cruzadas entre grupos	
7	Correcciones, deploy final y README	**Hito 2:** versión de entrega
8		
9	Presentación final	**Defensa oral**

La plantilla

No arrancás de cero. La plantilla **tienda-plantilla.zip** ya trae la estructura en capas, la página con sus estilos y el carrito, los controllers, las rutas, el script de las tablas y un README con el checklist. Ustedes completan las partes que conectan todo:

Clase 2	los models (productos, usuarios y pedidos) y el middleware que verifica al usuario
Clase 3	las funciones del front que llaman a la API con fetch

> ✓ **Lo que falta responde "501 Falta implementar".**
>
> Es a propósito: la tienda arranca desde hoy y vas viendo en pantalla qué te falta. Cada aviso dice en qué clase y paso se completa.

Paso 1 — Crear el proyecto en Supabase

Un proyecto por grupo. Lo crea un integrante y suma a los demás, así todos ven la misma base.

1. En el panel de Supabase, **New project**. Nombre: tienda-[rubro], por ejemplo tienda-libreria.
2. Región: **South America (São Paulo)**, la más cercana.
3. Generá la contraseña de la base y guardala en un lugar seguro: no se vuelve a mostrar.
4. Sumá a tus compañeros desde **Organization → Members → Invite**.
5. Entrá a **Project Settings → API Keys** y copiá la URL del proyecto y la **clave secreta** (secret key; en proyectos más viejos se llama service_role). Las vas a usar en el Paso 5.

> ⚠ **¿Por qué la clave secreta y no la pública?**
>
> Porque la usa solamente tu servidor. Con ella, Express puede leer y escribir todo, aunque las tablas estén cerradas al público. Es la clave más delicada del proyecto: jamás va al front, ni a GitHub, ni a una captura de pantalla.

Paso 2 — Crear las tablas

Los usuarios **no** los creás vos: Supabase Auth ya trae su tabla auth.users, con el email y la contraseña cifrada. Las tablas de la tienda son estas:

categorias	id, nombre	agrupar los productos

productos	id, nombre, descripcion, precio, stock, imagen_url, categoria_id, activo	el catálogo
perfiles	id (→ auth.users), nombre, rol	los datos del usuario que Auth no guarda
pedidos	id, usuario_id (→ auth.users), total, estado, creado	cada compra
pedido_items	id, pedido_id, producto_id, cantidad, precio_unitario	qué se compró en cada pedido

El script completo está en la plantilla, en **db/esquema.sql**. **Todavía no lo corras:** primero cambiá los datos de ejemplo por los de tu rubro (Paso 3). Mirá cómo está armada la tabla central:

```
create table productos (
 id bigint generated always as identity primary key,
 nombre text not null,
 descripcion text,
 precio numeric(10,2) not null check (precio >= 0),
 stock int not null default 0 check (stock >= 0),
 imagen_url text,
 categoria_id bigint references categorias(id),
 activo boolean not null default true
 );

 alter table productos enable row level security;
```

¿Qué hace cada parte?

check (precio >= 0)	la base rechaza precios o stock negativos, venga de donde venga el dato
references categorias(id)	la clave foránea: un producto solo puede apuntar a una categoría que exista
imagen_url	la dirección de la foto en Storage: la foto no se guarda en la tabla
activo	para "borrar" un producto sin perder los pedidos viejos que lo incluyen
enable row level security	cierra la tabla: con la clave pública no se puede leer ni escribir nada; solo entra tu backend con la secreta

⚠ **¿Para qué guardar precio_unitario en cada item, si el precio ya está en productos?** Porque los precios cambian. Si Ana compró 2 libros a $20.000 y el mes que viene suben a $25.000, su pedido tiene que seguir diciendo $40.000. precio_unitario es una foto del precio en el momento de la compra.

Paso 3 — Cargar los datos de tu rubro

Abrí **db/esquema.sql** y, al final, reemplazá los insert de ejemplo por las categorías y los productos de tu tienda. Por ejemplo, para una librería:

```
-- ===== DATOS DE LA TIENDA =====
insert into categorias (nombre) values
 ('Novelas'), ('Infantiles'), ('Útiles escolares');

insert into productos (nombre, descripcion, precio, stock, categoria_id) values  ('El Eternauta', 'Edición integral', 32000, 6, 1),  ('Rayuela', 'Bolsillo', 18000, 4, 1),  ('Cuentos de la selva', 'Ilustrado', 14500, 10, 2),  ('El principito', 'Tapa dura', 16000, 8, 2),  ('Cuaderno A4 rayado', '84 hojas', 3900, 40, 3),  ('Caja de lápices x12', 'Colores', 5200, 1, 3),  ('Mochila escolar', 'Con ruedas', 42000, 3, 3),  ('Resaltadores x4', 'Colores flúo', 2800, 25, 3);
```

Al menos 3 categorías	para que el filtro del catálogo tenga sentido
Al menos 8 productos	para que la tienda se vea como una tienda
Precios y descripciones creíbles	es lo primero que se ve en la defensa
Un producto con stock 1 o 0	para poder probar el "sin stock" en la Clase 2

Ahora sí: copiá **todo** el archivo, pegalo en el **SQL Editor** de Supabase y apretá **Run**. Después abrí el **Table Editor** y confirmá que estén las cinco tablas y tus productos.

> ⚠ **categoria_id es el número de la categoría.**
>
> Las categorías reciben los id 1, 2, 3 en el orden del insert. En el ejemplo, "Novelas" es la 1 y "Útiles escolares" es la 3. Si cambiás el orden, cambian los números.

Si ya corriste el script con los datos de ejemplo, borralos y reiniciá los id con esta línea, y después corré solo tus insert:

```
truncate productos, categorias restart identity cascade;
```

Paso 4 — Las fotos en Storage

La base guarda texto y números, no fotos. Las imágenes van a **Storage**, que guarda el archivo y te da una URL. En la tabla solo se guarda esa URL.

1. En el panel, **Storage → New bucket**. Nombre: productos. Activá **Public bucket**.
2. Nombrá cada foto con el **id** de su producto: 1.jpg, 2.jpg, 3.jpg... Mirá los id en el Table Editor. 3. Subí

las fotos al bucket arrastrándolas. Que sean livianas: menos de 500 KB cada una. 4. Corré este UPDATE en el SQL Editor, cambiando xxxx por el código de tu proyecto (está en la URL que copiaste en el Paso 1):

```
update productos
set imagen_url =
 'https://xxxx.supabase.co/storage/v1/object/public/productos/' || id || '.jpg';
```

||	une textos en SQL
... || id || '.jpg'	para el producto 3 arma .../productos/3.jpg: un solo UPDATE carga las URL de todos
Public bucket	cualquiera puede ver las fotos con la URL, sin clave: es lo que necesita el navegador

✓ **Probá una URL antes de seguir.**

Copiá el imagen_url de un producto desde el Table Editor y pegalo en el navegador. Si se ve la foto, está todo bien. Usá fotos propias o de bancos de imágenes libres, como Unsplash o Pexels.

Paso 5 — La plantilla corriendo

Descomprimí **tienda-plantilla.zip**, abrí la carpeta en VS Code y, en la terminal:

```
# 1. Crear el .env copiando el modelo (en Windows: copy .env.example .env) cp .env.example
.env

# 2. Abrir el .env y completar con la URL y la clave secreta del Paso 1
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_KEY=tu_clave_secreta

# 3. Instalar las dependencias y levantar el servidor
npm install
npm start
```

Abrí http://localhost:3000. Tendrías que ver la tienda con **tus categorías** en los botones del filtro, y un aviso en rojo:

```
Falta implementar: cargarProductos (Clase 3)
```

✓ **Si ves tus categorías y ese aviso, está todo bien conectado.**

Las categorías prueban que el servidor llega a tu base con tu clave. Los productos aparecen cuando completes el backend (Clase 2) y el front (Clase 3).

Antes de cerrar, cambiá el nombre de la tienda en **public/index.html** (el <title> y el <h1>) y, si querés, los colores del principio de **public/estilos.css**.

Checklist de cierre de la clase

Antes de irte, verificá cada punto. Lo que no llegue hoy queda de tarea: la Clase 2 arranca directo con el backend.

Grupo y rubro	anotados en la tabla del principio	
Proyecto creado	todos los integrantes entran al panel	
Cinco tablas	aparecen en el Table Editor	
Datos del rubro	3 categorías y 8 productos o más, uno con stock bajo	
Fotos	cada imagen_url abre su foto en el navegador	
Plantilla	localhost:3000 muestra tus categorías	
Seguridad	el .env existe y no aparece en ninguna captura ni chat	

Errores frecuentes y soluciones

insert or update on table "productos" violates foreign key constraint

✕ **Causa: Un producto tiene un categoria_id que no existe (por ejemplo, 4 cuando solo hay 3 categorías).**

✓ **Solución: Revisá los números de categoria_id contra el orden del insert de categorías (Paso 3).**

relation "categorias" already exists

✕ **Causa: Corriste el script dos veces: las tablas ya estaban creadas.**

✓ **Solución: No hace falta volver a crearlas. Si querés cambiar los datos, usá el truncate del Paso 3 y corré solo los insert.**

new row violates check constraint "productos_precio_check"

✕ Causa: Algún producto tiene precio o stock negativo, o escribiste el precio con punto de miles (20.000).

✓ Solución: Escribí los números sin puntos ni signo: 20000. Los decimales van con punto: 1500.50.

La foto no se ve: aparece la imagen rota

✕ Causa: El bucket no es público, el nombre del archivo no coincide con el id, o la foto es .png y el UPDATE dice .jpg.

✓ Solución: Marcá el bucket como Public y revisá que nombres y extensiones coincidan. Para un .png, corré el UPDATE de ese producto con .png.

En localhost:3000 no aparecen las categorías

✕ Causa: El .env tiene mal la URL o la clave, o le falta algún dato.

✓ Solución: Revisá el .env: sin espacios ni comillas, y con la clave secreta. Después reiniciá el servidor (Ctrl + C y npm start).

Cannot find module 'express'

✕ Causa: No corriste npm install, o lo corriste en otra carpeta.

✓ Solución: Corré npm install dentro de la carpeta tienda-plantilla, donde está el package.json.

Extensión opcional — para los que terminan antes

- Subí la plantilla a un repositorio de GitHub. Antes del primer push, confirmá que el .env **no** aparece en la lista de archivos: el .gitignore ya lo deja afuera.
- Agregá más productos, hasta 12 o 15, para que el catálogo se vea completo.
- Pensá qué extra le sumaría más valor a tu rubro y anotalo en el README: una librería con buscador, una panadería con "Mis pedidos"...

- Elegí la paleta de colores de tu tienda y cambiala en las variables de public/estilos.css.

> ⚠ **Recordá para el proyecto final**
>
> Todo lo de hoy ya es tu TP: la base, los productos y las fotos que cargaste son los que va a mostrar tu tienda online. El Hito 1 (Clase 4) pide la tienda publicada con el backend completo, así que cada clase que se atrasa se nota ahí.